

PRACTICAL-7

QNS:

Write a program that prints the addresses of code, global, static, heap, and stack variables. Analyze the Linux process address space layout and explain each memory segment.

```
GNU nano 8.7.1 memory.c *
#include <stdio.h>
#include <stdlib.h>

int global_initialized = 10;
int global_uninitialized;

void code_function()
{
    printf("Code address          : %p\n", (void *)code_function);
}

int main()
{
    static int static_initialized = 20;
    static int static_uninitialized;

    int stack_variable = 30;

    int *heap_variable = (int *)malloc(sizeof(int));
    *heap_variable = 40;

    printf("Linux Process Memory Addresses\n");
    printf("-----\n");

    code_function();

    printf("Global initialized address: %p\n",
        (void *)&global_initialized);

    printf("Global uninitialized addr : %p\n",
        (void *)&global_uninitialized);

    printf("Static initialized address: %p\n",
```

```

    (void *)&global_uninitialized);

    printf("Static initialized address: %p\n",
           (void *)&static_initialized);

    printf("Static uninitialized addr : %p\n",
           (void *)&static_uninitialized);

    printf("Heap address          : %p\n",
           (void *)heap_variable);

    printf("Stack address          : %p\n",
           (void *)&stack_variable);

    free(heap_variable);

    return 0;
}

```

```

priylvn@LAPTOP-NOJKE600:~$ cd fifo_project
priylvn@LAPTOP-NOJKE600:~/fifo_project$ nano memory.c
priylvn@LAPTOP-NOJKE600:~/fifo_project$ gcc memory.c -o memory
priylvn@LAPTOP-NOJKE600:~/fifo_project$ ./memory
Linux Process Memory Addresses
-----
Code address          : 0x5ba36917f1c9
Global initialized address: 0x5ba369182010
Global uninitialized addr : 0x5ba36918201c
Static initialized address: 0x5ba369182014
Static uninitialized addr : 0x5ba369182020
Heap address          : 0x5ba38a428010
Stack address         : 0x7ffc7e86ee3c
priylvn@LAPTOP-NOJKE600:~/fifo_project$ |

```

OBSERVATION:

The program was successfully executed in Ubuntu and the memory addresses of code, global, static, heap, and stack variables were displayed. The addresses showed that different types of data are stored in different regions of the Linux process address space. Global and static variables are stored in the data/BSS segments, dynamically allocated memory is stored in the heap, local variables are stored on the stack, and program

instructions are stored in the code segment. The exact addresses may vary between executions due to Linux Address Space Layout Randomization (ASLR).

QNS:

Make use of the `/proc//maps` file and memory analysis tools to study the memory organization of a running process.

Prepae a report explaining virtual memory mappings.

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int global_variable = 100;

int main()
{
    int stack_variable = 10;

    int *heap_variable = malloc(sizeof(int));
    *heap_variable = 20;

    printf("Process is running...\n");
    printf("PID: %d\n", getpid());

    printf("Address of code : %p\n", (void *)main);
    printf("Address of global variable : %p\n",
           (void *)&global_variable);
    printf("Address of heap variable : %p\n",
           (void *)heap_variable);
    printf("Address of stack variable : %p\n",
           (void *)&stack_variable);

    printf("\nKeep this program running and inspect /proc/PID/maps\n");

    while (1)
    {
        sleep(5);
    }

    free(heap_variable);
}
```